

Fundamentos e Busca - Inteligência Artificial

1- IA, Inteligência e Racionalidade

Considere os sistemas abaixo e para cada sistema, discuta em que sentido ele pode ou não ser considerado um sistema de Inteligência Artificial. Em seguida, explique a diferença entre agir racionalmente e agir de maneira semelhante a um ser humano.

1a. Um sistema de recomendação de filme.

Pode ser considerado um sistema de Inteligência Artificial, pois utiliza informações sobre os usuários e seus históricos para identificar padrões e recomendar filmes.

Agir Racionalmente: Recomendar filmes que tenham maior chance de agradar ao usuário.

Agir como um humano: Recomendar filmes de maneira semelhante à que uma pessoa faria, considerando preferências, opiniões e experiências.

1b. Um termostato que liga e desliga automaticamente o aquecimento.

Pode ser considerado um sistema automatizado, mas não necessariamente é um sistema de Inteligência Artificial, pois ele liga e desliga dependendo da temperatura.

Agir Racionalmente: Manter a temperatura próxima a de um valor desejado.

Agir como um humano: Tomar decisões sobre a temperatura de maneira semelhante a uma pessoa, considerando diferentes situações e preferências.

1c. Um aplicativo que calcula a menor rota entre duas pontas.

Pode ser considerado um sistema de Inteligência Artificial porque pode

utilizar algoritmos de busca para analisar diferentes possibilidades e encontrar uma rota que atenda aos objetivos, como chegar ao destino percorrendo a menor distância ou tendo o menor custo.

1d. Um chatbot capaz de manter uma conversação em linguagem natural.

Pode ser considerado um sistema de Inteligência Artificial porque é capaz de interpretar mensagens em linguagem natural e produzir respostas de acordo com o que foi dito pelo usuário. Nesse caso, o sistema precisa lidar com as informações da conversa para escolher respostas adequadas.

1e. Um filtro de spam que aprende com mensagens classificadas pelo usuário.

É um sistema de Inteligência Artificial porque aprende a partir de exemplos de receber mensagens classificadas pelo usuário como spam ou não spam, o sistema consegue identificar padrões e utilizar o que aprendeu para classificar novas mensagens.

2. PEAS e propriedades do ambiente

Um robô autônomo realiza a entrega de medicamentos em um hospital. Ele deve sair da farmácia, visitar os quartos indicados, evitar pessoas e obstáculos, respeitar áreas restritas e retornar à base.

1a. Descreva o ambiente da tarefa utilizando o modelo PEAS: medida de desempenho, ambiente, atuadores e sensores

Performance → levar medicamento para os pacientes certos

ambiente → hospital, pacientes, farmácia, quartos, pessoas

Atuadores → Rodas, motores, compartimento que abre para guardar os medicamentos

tilibra

Dimensões → Dimensões de proximidade, mapeamento do ambiente em tempo real.

1b. Classifique o ambiente quanto a: observabilidade, determinismo, estrutura episódica ou sequencial, dinamismo, discretização e quantidade de agentes.

Observabilidade → Parcialmente Observável.

Determinismo → Estocástico

Dependência → Sequencial

Dinâmica → Dinâmico

Continuidade → Contínuo

Quantidade de agentes → Multiagente

Conhecimento → Conhecido

1c. Há uma mudança no ambiente que obrigaria o agente a manter estado interno.

Se o robô, ao passar por um corredor, precisa lembrar quais quartos já visitou (entregou o medicamento) e quais ainda faltam.

Como o ambiente é parcialmente observável, o robô não consegue ver todos os quartos ao mesmo tempo, ele só percebe o que está imediatamente na sua frente ou ao alcance dos sensores naquele instante. Sem guardar estado interno, ele não teria como saber, ao voltar a passar por um corredor, se já entregou o medicamento da aquela sala ou não, arriscando repetir entregas ou pular quartos.

3. Considere um mundo com duas salas, A e B. Cada sala pode estar limpa ou suja. O agente percebe sua posição e se está de ida ou volta atual. As ações possíveis são Aspirar, deixar a lixeira e voltar e Parar.

A medida de desempenho concede +10 pontos para cada sala limpa ao final de cada passo, cobra -1 ponto por movimento e considera apenas os primeiros 20 passos.

1a. Um agente reflexivo simples que sempre aspira quando encontra sujeira e, caso contrário, muda de sala é necessariamente racional? Justifique.

Não, não é necessário racional. O agente reflexivo simples segue uma regra fixa (aspira se suja, senão troca de sala) sem levar em conta a existência de sujeira nem o custo acumulado, então ele continua se movendo entre as salas mesmo quando já estão limpas, gastando pontos com movimentos desnecessários (-1) quando a sujeira racional seria parar, o que reduz a pontuação total dentro das 20 passadas disponíveis.

1b. A inclusão do custo de movimento altera a estratégia do agente? Explique.

Sim, a inclusão do custo de movimento altera a melhor estratégia. Sem esse custo, mover-se sem necessidade não teria penalidade, então a veridica frequência dos movimentos importaria pouco. Com o custo de -1 por movimento, a estratégia ótima passa a evitar movimentos desnecessários: o agente deve se mover apenas quando a sala atual já está limpa e ainda há sujeira potencial na outra sala, e deve parar assim que ambas as salas estiverem limpas, já que continuar se movendo apenas reduz a pontuação sem gerar ganho adicional.

1c. Há vantagem em utilizar um agente baseado em modelo? Qual informação deveria ser armazenada?

Sim, há vantagem. Um agente baseado em modelo deveria armazenar as

estado (limpe / sujo) de cada sala e sua posição, evitando movimentos e verificações desnecessárias.

id. A ação racional pode depender do instante t em que o agente se encontra? dê um exemplo

Sim, perto do limite de 20 passos, a ação racional muda se vale a pena agir (aspirar) ou ainda gerar ganho líquido antes do tempo acabar, no último passo, mesmo sem necessidade não compensa mais.

4. Você possui três recipientes sem marcações, com capacidade de 8,5 e 3 litros. Há uma torneira com água ilimitada. É permitido encher um recipiente, esvaziá-lo ou transferir água de um recipiente para outro até que a origem fique vazia ou o destino fique cheio. O objetivo é obter exatamente 4 litros em algum recipiente. Formule o problema de busca especificando:

a. Representação de um estado.

Quantidade de água em cada recipiente

b. Estado inicial.

Os três recipientes estarão vazios.

c. Ações possíveis.

Começar a encher os recipientes até sua capacidade máxima, esvaziar completamente um recipiente ou transferir água de um recipiente para o outro até que a origem fique vazia ou o destino fique cheio.

d. Modelo de transição.

A origem ficar vazia ou o destino cheio.

1e. Teste de objetivo.

Obter exatamente 4 litros em algum recipiente.

2f. Custo do passo e custo do caminho.

Custo passo = 1 para cada ação (encher, transferir ou esvaziar).

Custo caminho = soma dos custos de passo ao longo da sequência de ações.

Por fim, procura pelo menor uma sequência válida de estados que seja a uma solução.

$(0,0,0) \rightarrow (0,5,0) \rightarrow (0,2,3) \rightarrow (0,2,0) \rightarrow (0,0,2) \rightarrow (0,5,2) \rightarrow (0,4,3)$.

5. Abstração e tamanho do espaço de estados

Um robô se desloca em um labirinto retangular com M linhas e N colunas. Em cada célula livre, ele pode estar orientado para Norte, Sul, Leste ou Oeste. Duas ações são: girar à esquerda, girar à direita e avançar uma célula.

1a. Escreva uma representação de estado adequada.

Um estado pode ser representado pela tupla (linha, coluna, orientação), onde (linha, coluna) indica a célula onde o robô está no labirinto e orientação (norte, sul, leste, oeste) indica a direção para a qual ele está voltando.

1b. Desconsiderando paredes, qual é o número máximo de estados possíveis?

Quatro estados, norte - sul - leste - oeste

1c. Duplique a rotação que o robô usa precisa tomar decisões em N inter.

veias do labirinto e passa percorrer automaticamente um corredor até a próxima interseção. Responda uma mesa representação.

Em vez de se "voto" pensar em cada célula do labirinto (o que dá muitos estados), ele só precisa pensar nas interseções, porque nos corredores ele simplesmente anda reto até a próxima interseção, não há decisão a tomar ali. O novo estado é só a qual interseção ele está, para onde está идando).

1d. Explique por que uma boa abstração pode diminuir drasticamente o custo da busca sem alterar a solução do problema.

Uma boa abstração reduz o número de estados considerados (lugares onde pntes sem decisão, como corredores), sem eliminar soluções reais. A busca fica mais rápida, mas o caminho final continua correto ao ser expandido de volta ao problema original.

6. Estado, nó e árvore de busca.

Responda de forma objetiva, utilizando exemplos quando necessário.

1a. Qual é a diferença entre estado do mundo, descrição de estado e nó de busca?

Estado do mundo = configuração real e completa do ambiente em determinado momento (posição exata de todos os objetos).

Descrição de estado = Representação abstrata usada pelo agente para capturar esse estado.

Nó de busca = É a estrutura de dados usada durante o algoritmo de busca, que contém a descrição de estado mais informações adicionais do processo de busca, como o nó pai, a ação que levou ali e o custo do caminho.

1b. duas mais diferentes de uma variável podem representar o mesmo estado? Explique.

Sim, duas mais diferentes podem representar o mesmo estado, porque um nó é definido pelo caminho percorrido até ele, e estados diferentes de caminhos podem levar ao mesmo lugar.

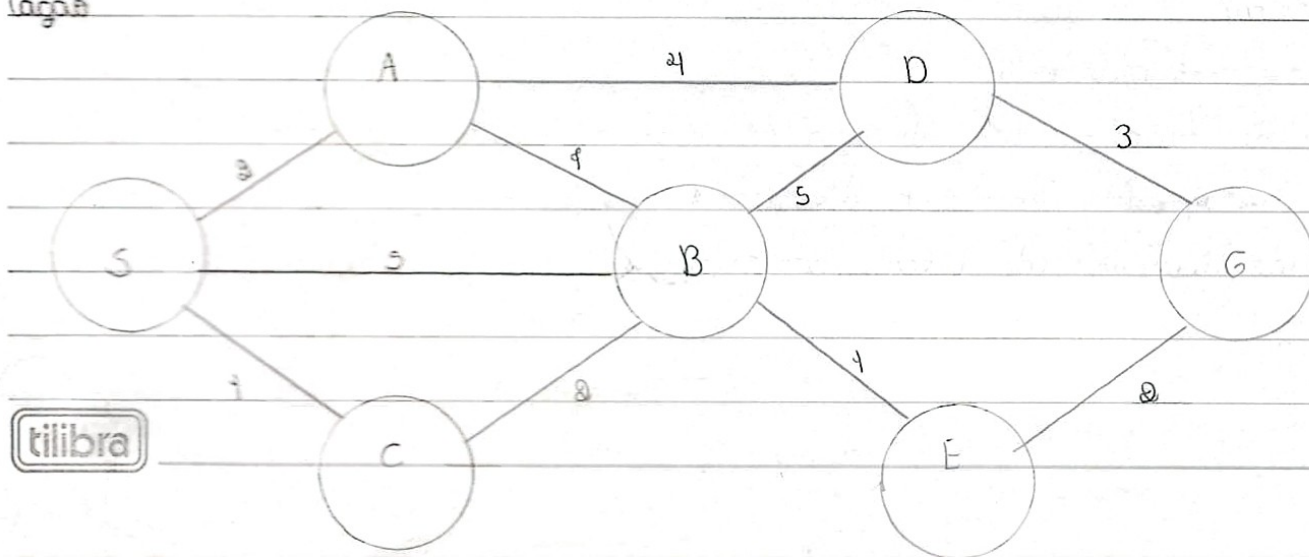
1c. Um espaço de estados finito sempre gera uma árvore de busca finita? Justifique com um exemplo ou contraexemplo.

Não, um espaço de estados finito pode gerar uma árvore de busca infinita, porque a árvore de busca representa caminhos, e caminhos podem repetir estados indefinidamente quando há ciclos. Por exemplo, com apenas dois estados A e B conectados nos dois sentidos, a árvore fica $A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \dots$, infinita, mesmo com só dois estados no espaço.

1d. Por que a detecção de estados repetidos é importante em busca em grafos?

Para evitar expandir o mesmo estado por caminhos ou ciclos diferentes, economizando tempo e memória e impedindo loops infinitos, já que cada estado é expandido no máximo uma vez.

4. Considere o grafo abaixo. O estado inicial é S e objetivo é G. As arestas são indirecionadas e o número em cada aresta representa o custo da aresta.



Para cada algoritmo abaixo, represente a ordem de expansão de nós e o caminho retornados. Em caso de empate, use ordem alfabética.

* **a.** Busca em largura (BFS) $S \rightarrow$ Primeiro que entra primeiro que sai
Frenteira: S, A, B, C, D, E, G

Expandiu: S, A, B, C, D, E, G

Caminho Armazenado - $S \rightarrow$ S

S

A $\rightarrow$ S

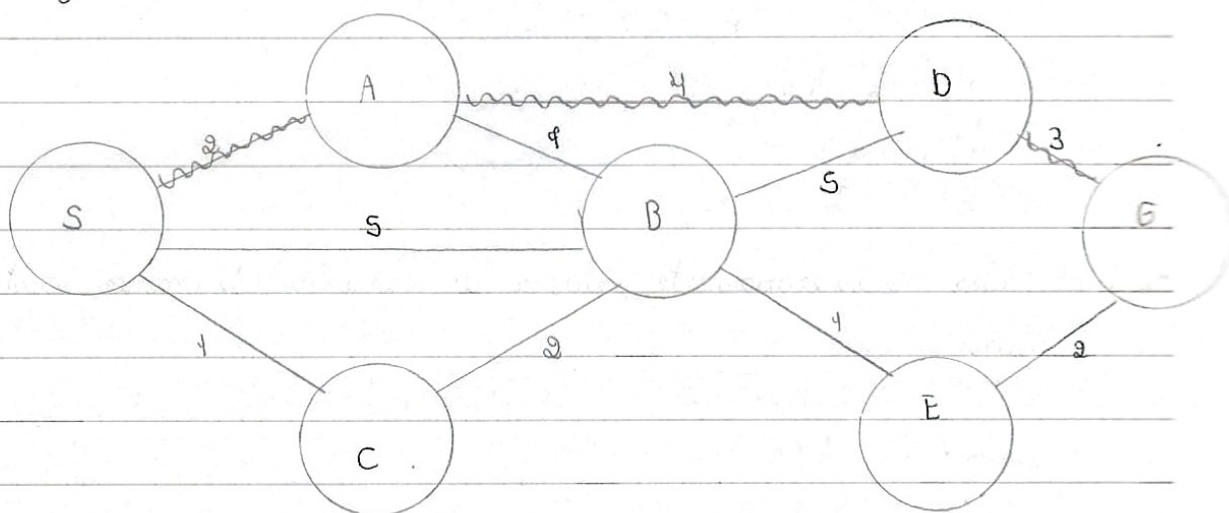
B $\rightarrow$ S

C $\rightarrow$ S

D $\rightarrow$ A

E $\rightarrow$ B

G $\rightarrow$ D



Caminho: $S \rightarrow A \rightarrow D \rightarrow G$

Custo: 3 arestas

* **b.** Busca em profundidade (DFS) $S \rightarrow$ Primeiro que entra último que sai
Frenteira: S, A, B, C, D, E, G

Expandiu: S, C, B, E, G

Caminho Armazenado - $S \rightarrow$ S

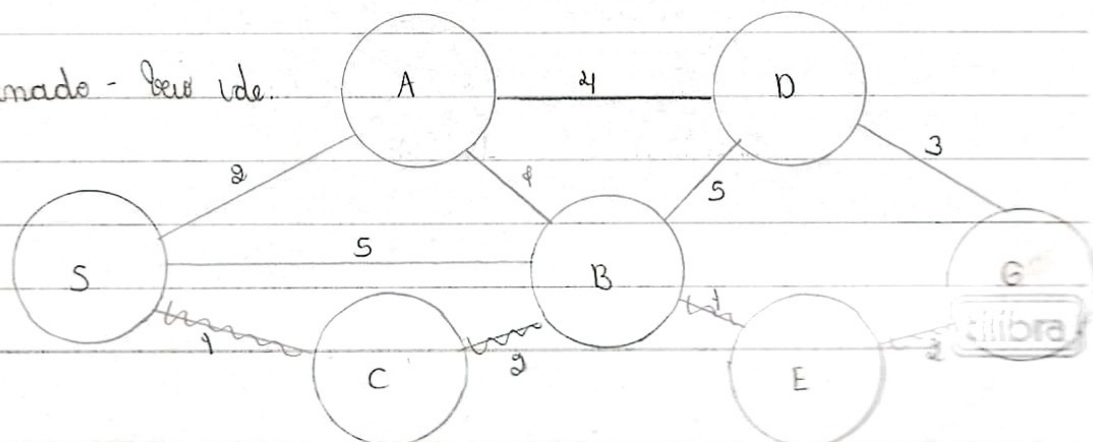
S

C $\rightarrow$ S

B $\rightarrow$ S

E $\rightarrow$ B

G $\rightarrow$ E



Laminha: $S \rightarrow C \rightarrow B \rightarrow E \rightarrow G$

Custo: 4 arestas

* 1c. Busca de custo uniforme (UCS) Custo = 6

Expandir fronteira após expandir

S C_1, A_2, B_3

C_1 A_2, B_3

A_2 B_3, D_4

B_3 E_1, D_6

E_1 D_6, G_6

D_6 G_6

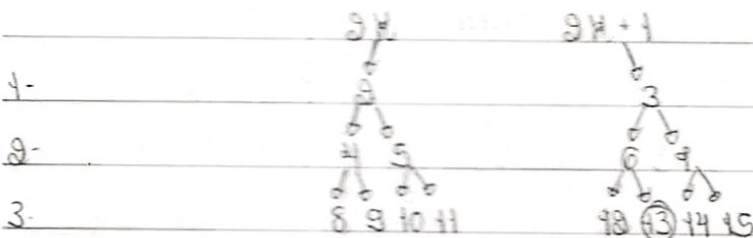
G_6 Algoritmo

1d. Qual deles necessariamente retorna o caminho de menor custo neste grafo? Justifique.

A busca em largura, pois o custo dela é só de 3 arestas

8. Considere uma árvore em que o estado inicial é 1 e cada estado inteiro positivo k gera, nesta ordem, os sucessores $2k$ e $2k+1$. O objetivo é alcançar o estado 13.

1a. Desenhe a árvore até a profundidade 3.

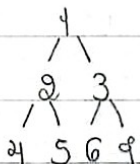


1b. Execute uma busca em profundidade limitada (DLB) com limite $d=2$. O objetivo é encontrado? Pilha

$P=0$

$P=1$

$P=2$



Fronteira = ~~1~~ 2 3 ~~6~~ ~~4~~ ~~5~~ ~~7~~

Expandido = 1 3 7 6 2 5 4

Laminha Armazenado - seu id:

1

3 $\rightarrow$ 1

7 $\rightarrow$ 3

6 $\rightarrow$ 3

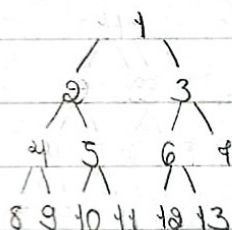
2 $\rightarrow$ 1

5 $\rightarrow$ 2

4 $\rightarrow$ 2

Não, pois o objetivo 13 não está localizado

1c. Execute a busca em profundidade iterativa (IDS) até encontrar o objetivo. Mostre quais nós são visitados em cada limite. Pila



Fronteira: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13

Expandiu: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13

Caminho Imaginado - tree de

1

2 → 1

3 → 1

4 → 2

5 → 2

6 → 3

7 → 3

8 → 4

9 → 4

10 → 5

11 → 5

12 → 6

13 → 6

id. Compare BFS e IDS quanto à completude, optimalidade para custos unitários, tempo e memória

	Busca em Largura	Busca em Profundidade
Completude	Sim (se o fator de ramificação for finito)	Sim (se o fator de ramificação for finito)
Optimalidade	Sim (encontra o caminho mais curto / menos profundo)	Sim (encontra a solução na menor profundidade possível)
Tempo	$O(b^d)$	$O(b^d)$ (apesar das re-visitadas, a última camada domina a soma)

Memória	$O(b^{ld})$ (armazena todos os nós na memória)	$O(b^{ld})$ (utiliza apenas a memória necessária para a pilha da DFS)
---------	--	---

9. Comparação entre estratégias de busca.

Considere um problema com fator de ramificação b , solução mais curta com profundidade l , profundidade máxima m e custos de passos positivos limitados uniformemente por $\epsilon > 0$.

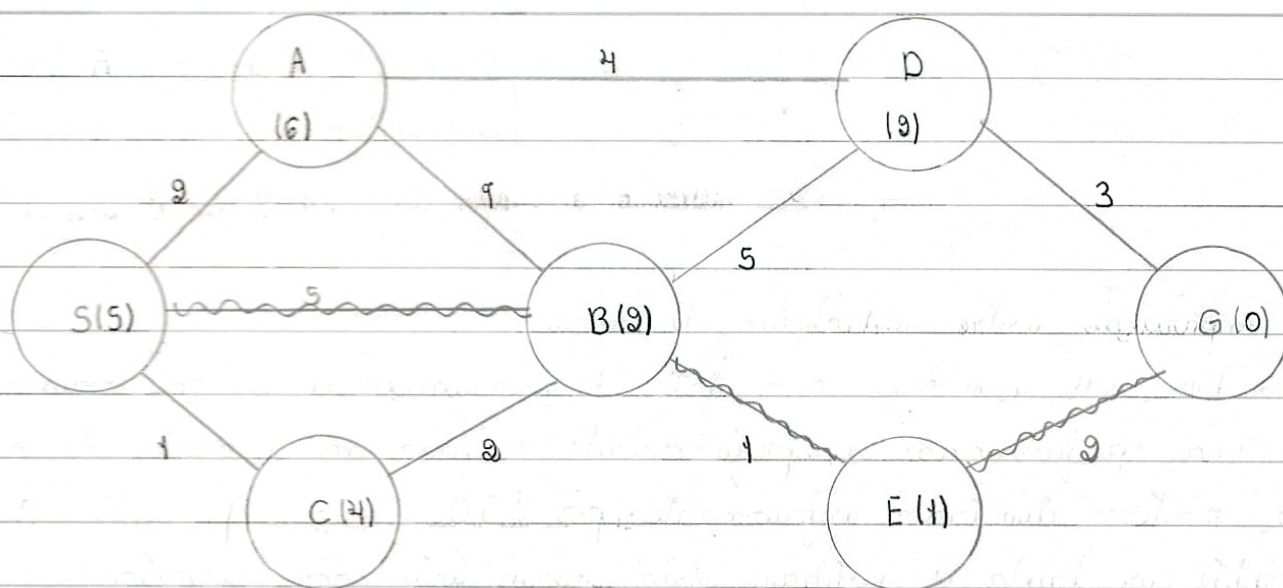
Complete a tabela e justifique pelo menos três das respostas.

Algoritmo	Completo?	Ótimo?	Tempo	Memória
BFS	Sim	Sim	$O(b^{ld})$	$O(b^{ld})$
UCS	Sim	Sim	$O(b^{ld} + \frac{C}{\epsilon})$	$O(b^{ld} + \frac{C}{\epsilon})$
DFS	Não	Não	$O(b^m)$	$O(b^{ld})$
IDF	Sim	Sim	$O(b^{ld})$	$O(b^{ld})$
Busca Gulosa	Não	Não	$O(b^m)$	$O(b^m)$
A*	Sim	Sim	Depende da heurística	Depende da heurística

10. Utilize meramente o grafo do Exercício 9 e considere os seguintes valores heurísticos para os vértices G:

$h(u)$	5	6	2	4	2	1	0
	S	A	B	C	D	E	G

11. Execute a Busca Gulosa, mostrando em cada passo o nó escolhido para expansão.

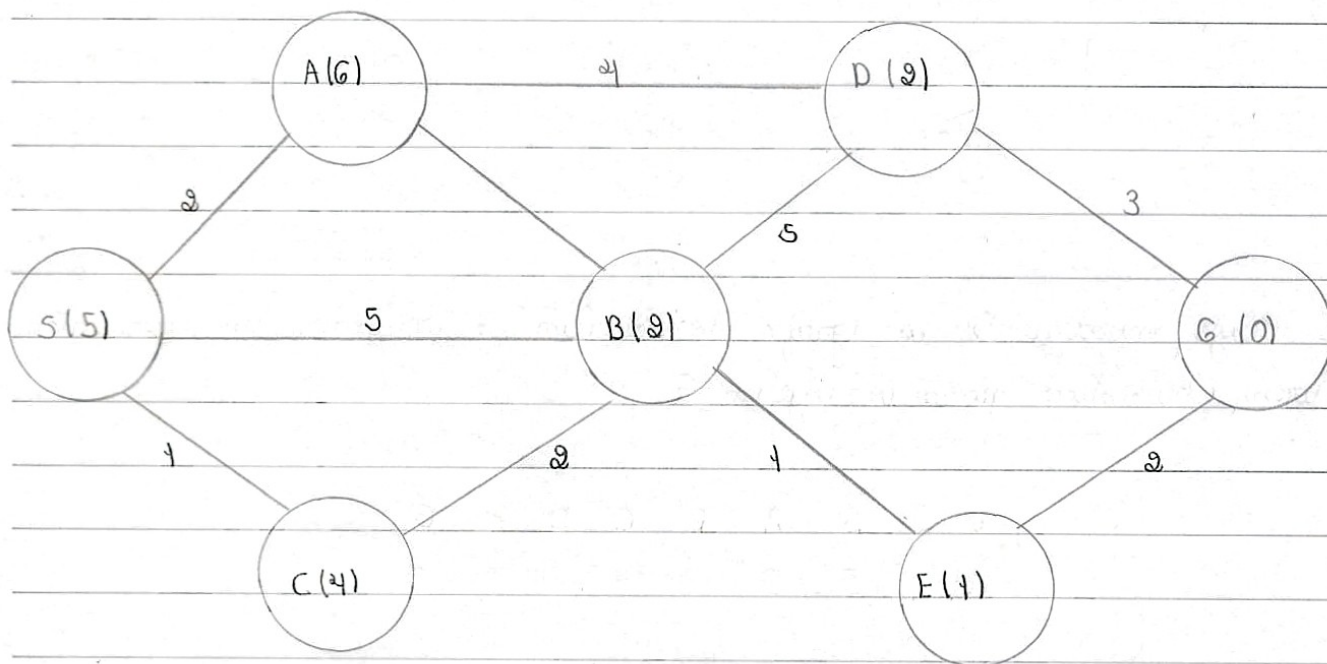


$S(5) \rightarrow B(2) \rightarrow E(1) \rightarrow G(0)$

$$= 5 + 2 + 1 + 0$$

$$= 8$$

1b. Ejecute A*, calculando $f(n) = g(n) + h(n)$ para más relevantes.



$$S \rightarrow C = 4$$

$$\text{Costo Total} = 6$$

$$C \rightarrow B = 2$$

$$B \rightarrow E = 1$$

$$E \rightarrow G = 2$$

1c. Os dois algoritmos retornam necessariamente a mesma redução? Explique conceitualmente.

Não, enquanto a busca gulosa retorna o custo total de 8, a busca A^* retorna 6.

1d. Qual é a diferença essencial entre os critérios de escolha utilizados pela busca gulosa e por A^* .

Gulosa usa $h(n)$ (estimativa restante, ignora o custo já gasto), A^* usa $f(n) = g(n) + h(n)$, equilibrando custo já percorrido com estimativa restante, por isso A^* garante o caminho ótimo e Gulosa não.

11. Admissibilidade e Consistência

Considere novamente o grafo do Exercício 9. Para cada uma das heurísticas abaixo, classifique-a como **admissível** e **consistente**, **admissível mas inconsistente** ou **não admissível**. Justifique matematicamente; quando uma propriedade falhar, represente um estado ou uma aresta que viole-a como contraexemplo.

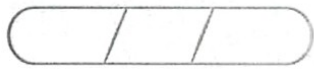
	S	A	B	C	D	E	G
H1	6	6	3	4	3	2	0
H2	9	9	3	5	3	2	0
H3	6	9	3	5	3	2	0

Use as condições

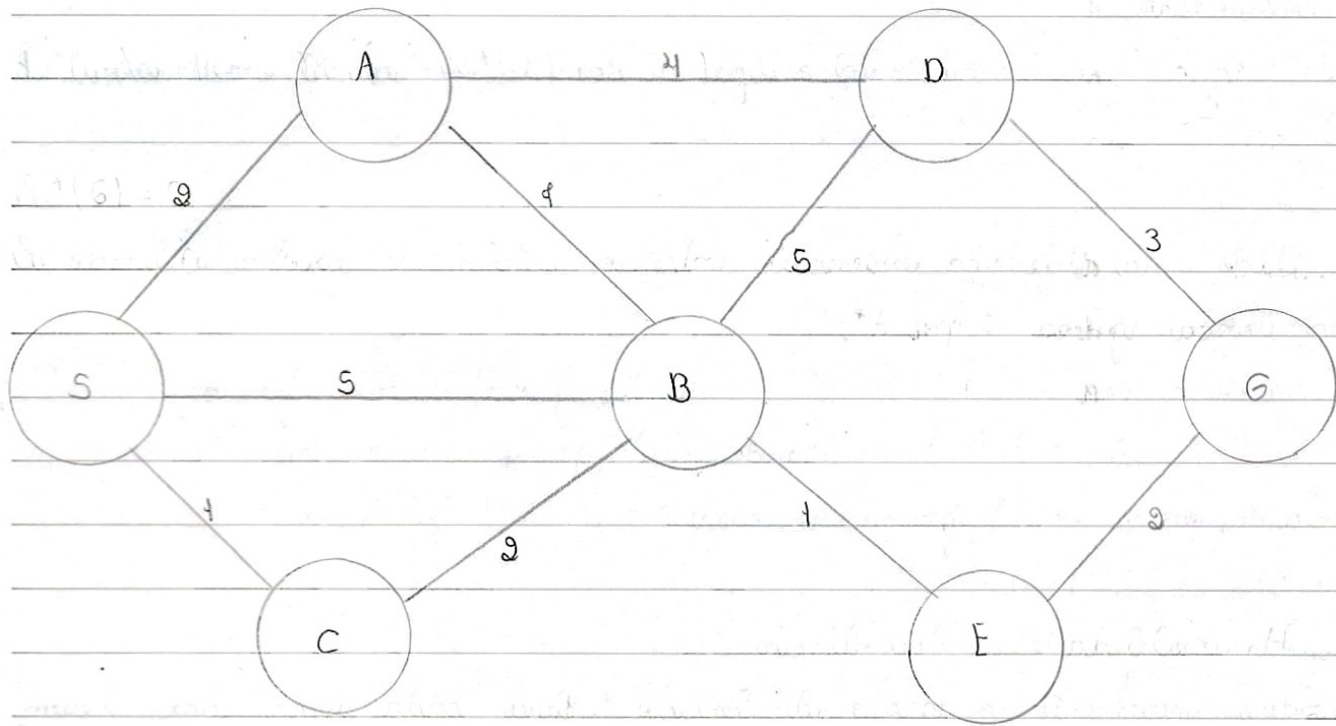
$$h(n) \leq h^*(n)$$

para admissibilidade

$$h(n) \leq C(n, n') + h(n')$$



para consistência.



Para obter os valores $h^*(v)$

$$h^*(G) = 0$$

$$h^*(E) = 2$$

$$h^*(D) = 3$$

$$h^*(C) = 5$$

$$h^*(B) = 3$$

$$h^*(A) = 4$$

$$h^*(S) = 6$$

Admissibilidade H_1

$H_1 \rightarrow$	S	A	B	C	D	E	G
	6	6	3	4	3	2	0

$$sh_1(m) \leq sh^*(m)$$

$$S = 6 \leq 6 \quad OK$$

$$A = 6 \leq 9 \quad OK$$

$$B = 3 \leq 3 \quad OK$$

$$C = 4 \leq 5 \quad OK$$

$$D = 3 \leq 3 \quad OK$$

$$E = 2 \leq 2 \quad OK$$

$$G = 0 \leq 0 \quad OK$$

Consistência H1

$$sh_1(S) \leq w(S, C) + sh_1(C)$$

$$6 \leq 4 + 4$$

$$6 \leq 5$$

Admissível mas inconsistente.

Admissibilidade H2

$$H2 \rightarrow S \quad A \quad B \quad C \quad D \quad E \quad G$$

$$9 \quad 9 \quad 3 \quad 5 \quad 3 \quad 2 \quad 0$$

$$sh_2(m) \leq sh^*(m)$$

$$S = 9 \leq 6 \quad ERRO$$

$$A = 9 \leq 9 \quad OK$$

$$B = 3 \leq 3 \quad OK$$

$$C = 5 \leq 5 \quad OK$$

$$D = 3 \leq 3 \quad OK$$

$$E = 2 \leq 2 \quad OK$$

$$G = 0 \leq 0 \quad OK$$

Consistência

$$h_2(S) \leq (S, C) + h_2(C)$$

$$4 \leq 1 + 5$$

$$4 \leq 6$$

Não admissível e não consistente

Admissibilidade H_3

$$H_3 \rightarrow S \quad A \quad B \quad C \quad D \quad E \quad G$$

$$6 \quad 4 \quad 3 \quad 5 \quad 3 \quad 2 \quad 0$$

$$h_3(m) \leq h^*(m)$$

$$S = 6 \leq 6 \quad OK$$

$$A = 4 \leq 4 \quad OK$$

$$B = 3 \leq 3 \quad OK$$

$$C = 5 \leq 5 \quad OK$$

$$D = 3 \leq 3 \quad OK$$

$$E = 2 \leq 2 \quad OK$$

$$G = 0 \leq 0 \quad OK$$

Consistência

$$h_3(S) \leq (S, A) + h_3(A)$$

$$h_3(S) \leq (S, B) + h_3(B)$$

$$6 \leq 2 + 4$$

$$6 \leq 5 + 3$$

$$6 \leq 9$$

$$6 \leq 8$$

$$h_3(S) \leq (S, C) + h_3(C)$$

$$h_3(A) \leq (A, D) + h_3(D)$$

$$6 \leq 1 + 5$$

$$4 \leq 4 + 3$$

$$6 \leq 6$$

$$4 \leq 4$$

$$h_3(A) \leq (A, B) + h_3(B)$$

$$4 \leq 4 + 3$$

$$4 \leq 10$$

$$h_3(C) \leq (C, B) + h_3(B)$$

$$5 \leq 2 + 3$$

$$5 \leq 5$$

$$h_3(B) \leq (B, E) + h_3(E)$$

$$3 \leq 1 + 2$$

$$3 \leq 3$$

$$h_3(B) \leq (B, D) + h_3(D)$$

$$3 \leq 5 + 3$$

$$3 \leq 9$$

$$h_3(D) \leq (D, G) + h_3(G)$$

$$3 \leq 3 + 0$$

$$3 \leq 3$$

$$h_3(E) \leq (E, G) + h_3(G)$$

$$2 \leq 2 + 0$$

$$2 \leq 2$$

Admissível e consistente.

12. Heurística para o 8-puzzle

Considere o estado inicial e o objetivo abaixo, em que os símbolos vazios em que os símbolos vazios representam a posição vazia.

Inicial

2	8	3
4	6	4
9		5

Objetivo

4	2	3
8		4
9	6	5

Calcule para o estado inicial:

10. H: número de peças fora da posição correta;

Posição	Inicial	Objetivo	Seu de Lugar
(1,1)	2	4	Sim
(1,2)	8	2	Sim
(1,3)	3	3	Não
(2,1)	4	8	Sim
(2,2)	6		Sim
(2,3)	4	4	Não
(3,1)	4	4	Não
(3,2)		6	Não conta (vazio)
(3,3)	5	5	Não

4 peças na posição errada

b. H2: soma das distâncias de Manhattan das peças até suas posições no objetivo

Peça	Posição Inicial	Posição Objetivo	Distância
2	(1,1)	(1,2)	$1-1 + 1-2 = 1$
8	(1,2)	(2,1)	$1-2 + 2-1 = 2$
4	(2,1)	(1,1)	$2-1 + 1-1 = 1$
6	(2,2)	(3,2)	$2-2 + 3-2 = 1$

$$1+2+1+1 = 5$$

c. Explique por que ambos são heurísticos admissíveis

H1 $\rightarrow$ É admissível porque cada peça errada precisa de pelo menos um movimento para ir para o lugar certo.

H2 $\rightarrow$ É admissível porque calcula a menor distância em linha reta de cada peça até seu destino.

13. Um veículo autônomo se desloca em uma malha $M \times N$ em cada ação ele pode mover uma célula na horizontal ou vertical, com custo 1. Algumas células são bloqueadas por obstáculos. O objetivo é chegar de (x_s, y_s) a (x_g, y_g) .

1a. Proponha uma heurística não trivial para A^* .

$h(m) = |x - x_g| + |y - y_g|$ onde (x, y) é a posição atual do veículo e (x_g, y_g) é a posição objetivo. Essa heurística estima o número mínimo de movimentos (horizontais + verticais) necessários para chegar ao objetivo, ignorando a existência de obstáculos, já que, sem obstáculos, esse seria exatamente o número de passos necessários, pois cada ação move uma célula na horizontal ou vertical com custo 1.

1b. Demonstre que sua heurística é admissível.

A distância de Manhattan é o custo mínimo sem obstáculos, como obstáculos só podem aumentar o custo real (nunca diminuir), $h(m)$ nunca superestima $h^*(m)$, logo é admissível.

1c. A remoção de algumas arestas do grafo, representando novos obstáculos, compromete a admissibilidade? Explique.

Não compromete, pois remover arestas só pode aumentar o custo real (nunca diminuir), então $h(m)$ continua $\leq h^*(m)$, permanece admissível, só fica menos informativa.

1d. É se forem adicionadas arestas que conectam células não adjacentes com custo 1? A mesma heurística continua admissível? Justifique.

Não necessariamente. Arestas com custo 1 entre células não adjacentes podem tornar o custo real menor que a distância de Manhattan calculada (que assume só movimentos entre células vizinhas), fazendo

$h(m) > h^*(m)$ em alguns casos, dependendo da admissibilidade.

12. Explique por que uma heurística inadmissível pode, em determinadas aplicações, ainda ser útil na prática.

Ela pode ser útil porque, mesmo sem garantir a solução ótima, costuma expandir muito menos nós, encontrando soluções boas rapidamente, com valores em problemas grandes onde a otimalidade é mesmo um parâmetro que a velocidade.

14. Sabrinho brinca com profundidade e brinca com largura.

Pensemos no labirinto abaixo. A célula A representa o estado inicial, a célula B representa o objetivo e as células pretas são obstáculos. Em cada estado, considere as ações válidas na seguinte ordem de prioridade:

Direita $\rightarrow$ Para $\rightarrow$ Esquerda $\rightarrow$ Baixo

	0	1	2	3	4	5	6	7
0								B
1								
2								
3								
4								
5								
6								
7	A							

15. Faça a busca em profundidade (DFS). Numere no labirinto a ordem em que os estados são expandidos e indique o caminho encontrado de A até B.

	0	1	2	3	4	5	6	7
0								19
1								16
2								15
3							13	14
4							12	
5							11	10
6								9
7	A	B	C	D	E	F	G	H

Filha $\rightarrow$ primeiro a entrar último a sair.

Um ramo só

$(7,0) \rightarrow (7,1) \rightarrow (7,2) \rightarrow (7,3) \rightarrow (7,4) \rightarrow (7,5) \rightarrow (7,6) \rightarrow (7,7) \rightarrow (6,7) \rightarrow (5,7) \rightarrow (5,6) \rightarrow (4,6) \rightarrow (3,6) \rightarrow (3,7) \rightarrow (2,7) \rightarrow (1,7)$

1b. Quantas células foram expandidas pela DFS? Quantos passos existem no caminho solução retornado.

Expandidas = 19 células

Passos = $19 - 1 = 18$ passos

1c. A solução encontrada pela DFS é necessariamente ótima? Explique.

Não, a DFS explora o ramo mais profundo primeiro sem comparar caminhos por tamanho, ela só busca achar o objetivo, não o caminho mais curto, então pode gerar uma solução bem mais longa que a ótima.

1d. Execute agora a busca em profundidade (BFS), usando a mesma ordem de ações. Numere a ordem de expansão e indique o caminho encontrado.

	0	1	2	3	4	5	6	7
0			29					³⁸ B
1	18	21	24	28	32	35		37
2	15				27			36
3	11				23	26	31	34
4	8		14		20		33	
5	5		10		17		30	25
6	3		9		13			22
7	A	2	7	6	9	12	16	19

ida → primeiro a entrar primeiro a sair
 volta mesmo

12. Quantas células foram expandidas pela BFS? Quantas passos existem no caminho da redução.

Células → 38 células

Passos → 17 passos

13. Compare DFS e BFS neste labirinto quanto ao caminho encontrado, número de expansões, memória utilizada e garantia de optimalidade.

	DFS	BFS
Caminho Encontrado	16	14
Número de Expansão	16	39
Memória	5	5
Garantia de Optimalidade	Não	Sim

15. Labirinto busca gulosa e A*.

Utiliza o mesmo labirinto do exercício anterior. Considere como heurística a distância de Manhattan entre uma célula (X, Y) e o objetivo (XB, YB).

$$h(m) = |X_m - X_B| + |Y_m - Y_B|$$

Cada movimento válido para uma célula adjacente possui custo 1. Em caso de empate, mantenha a prioridade de ações: direita, cima, esquerda, baixo.

1a. Recrie a **busca gulosa**, escolhendo para expansão o estado com menor $h(m)$. Mostre a ordem de expansão e o caminho retornado.

	0	1	2	3	4	5	6	7
0			5					8
1	8	9	6	5	4	3		1
2	9				5			2
3	10				6	5	4	3
4	11		9		4		5	
5	12		10		8		6	5
6	13		11		9			6
7	A	13	10	11	10	9	8	7

1b. Quantos estados foram expandidos pela busca gulosa? Qual é o comprimento do caminho encontrado?

Estados Expandidos = 19

Comprimento do caminho = 16

1c. A busca gulosa é garantidamente ótima neste problema? Justifique com base no critério utilizado para escolher o próximo estado.

Não, porque ela escolhe o próximo nó apenas com base em $h(m)$, ignorando o custo percorrido. Isso significa que ela pode se deixar levar por um caminho que parece mais perto do objetivo, mesmo que na prática seja mais longo.

id. execute w algoritmo A*, utilizando

$$f(m) = g(m) + h(m)$$

	0	1	2	3	4	5	6	7
0			5+9 14					B
1	8+6 14	9+7 14	6+8 14	5+9 14	4+10 14	3+11 14		1+13 14
2	9+5 14				5+11 16			2+12 14
3	10+4 14				6+8 14	5+9 14	4+10 14	3+11 14
4	11+3 14		8+5 14		9+7 14		5+11 16	
5	13+2 14		10+4 14		8+6 14		6+10 16	5+9 14
6	13+1 14		11+3 14		9+5 14			6+8 14
7	A	13+1 14	12+2 14	11+3 14	10+4 14	9+5 14	8+6 14	7+7 14

	$g(m)$	$h(m)$	$f(m)$
(7,0)	0	14	14
(7,1)	1	13	14
(7,2)	2	12	14
(7,3)	3	11	14
(7,4)	4	10	14
(6,4)	5	9	14
(5,4)	6	8	14
(4,4)	7	7	14
(3,4)	8	6	14
(3,5)	9	5	14
(3,6)	10	4	14
(3,7)	11	3	14
(2,7)	12	2	14
(1,7)	13	1	14
(0,7)	0	0	14

10. O caminho retornado por A^* é ótimo? Explique por que a distância de Manhattan é uma heurística admissível para este labirinto. Além disso, A^* é ótimo (custo 14). A Manhattan é admissível pois ela mede a distância ideal sem obstáculos, e como paredes não podem aumentar o caminho real, a estimativa nunca passa do valor real.

11. A distância de Manhattan também é consistente neste problema? Justifique usando a relação

$$dh(l, n) \leq c(l, m) + dh(m, n)$$

$$dh(4, 3) = (4-1) + (3-0) = 6$$

$$dh(0, 1) = (0-1) + (1-0) = 1$$

$$dh(5, 3) = (5-1) + (3-0) = 5$$

$$dh(1, 1) = (1-1) + (1-0) = 1$$

$$6 \leq 5 + 1$$

$$1 \leq 1 + 1$$

$$6 \leq 6$$

$$1 \leq 1$$

$$dh(1, 2) = (1-1) + (2-0) = 2$$

$$dh(1, 1) = (1-1) + (1-0) = 1$$

$$2 \leq 1 + 1$$

$$2 \leq 2$$

12. Compare, para este mesmo labirinto, DFS, BFS, Gulosa e A^* . Complete a tabela.

	Estados Expandidos	Passos de Solução	Última
DFS	19	16	Não
BFS	38	14	Sim
Gulosa	19	16	Não
A^*	36	14	Sim

Por fim, explique por que dois algoritmos podem encontrar o mesmo caminho, redução e, ainda assim, expandir quantidades muito diferentes de estados.

Os algoritmos podem achar o mesmo caminho, mas expandir números diferentes de estados, porque isso depende de quanta informação cada um usa para guiar a busca, quanto menos informação, mais expansões desperdiçadas em locais sem saída.

16. Implementação de algoritmo de busca

Considere o grafo abaixo representado em Python. Cada tupla possui o formato (vizinho, custo).

grafo = {

 'S': [(A, 0), (B, 5), (C, 1)],

 'A': [(S, 0), (D, 4), (B, 4)],

 'B': [(S, 5), (A, 4), (C, 2), (E, 1)],

 'C': [(S, 1), (B, 2), (D, 5)],

 'D': [(A, 4), (C, 5), (G, 3)],

 'E': [(B, 1), (G, 0)],

 'G': [(D, 3), (E, 0)].

}

heurística = {

 'S': 5,

 'A': 6,

 'B': 2,

 'C': 4,

 'D': 2,

 'E': 1,

 'G': 0.

O estado inicial é S e o objetivo é G. Implemente em Python os seguintes algoritmos:

1a. Busca em largura (BFS)

Atenção: a função deve retornar

- o caminho encontrado;
- a ordem em que os estados foram expandidos;
- o número total de estados expandidos.

```
from collections import deque
```

```
def BFS(grafo, inicio, objetivo):
```

```
    fila = deque([inicio, [inicio]])
```

```
    visitados = {inicio}
```

```
    ordem_expansao = []
```

```
    while fila:
```

```
        atual, caminho = fila.popleft()
```

```
        ordem_expansao.append(atual)
```

```
        if atual == objetivo:
```

```
            return caminho, ordem_expansao, len(ordem_expansao)
```

```
    return None, ordem_expansao, len(ordem_expansao)
```

```
    for vizinho, custo in grafo[atual]:
```

```
        if vizinho not in visitados:
```

```
            visitados.add(vizinho)
```

```
            fila.append((vizinho, caminho + vizinho))
```

```
    return None, ordem_expansao, len(ordem_expansao)
```

```
caminho, ordem, total = bfs(grafo, 'S', 'G')
```

19. Busca gulosa utilizando $h(n)$ para selecionar o próximo estado

```
import heapq

def gulosa (grafo, h, inicio, objetivo):
    contador = 0
    frontier = [(h[inicio], contador, inicio, [inicio])]
    visitados = set()
    ordem = []
    while frontier:
        _, _, atual, caminho = heapq.heappop(frontier)
        if atual == objetivo:
            return caminho, ordem
        if atual in visitados:
            continue
        visitados.add(atual)
        ordem.append(atual)
        if atual == objetivo:
            return caminho, ordem
        for vizinho, custo in grafo[atual]:
            if vizinho not in visitados:
                contador += 1
                heapq.heappush(frontier, (h[vizinho], contador, vizinho, caminho + [vizinho]))
    return None, ordem

caminho, ordem = gulosa(grafo, h, 'S', 'G')
```

10. Busca A^* , utilizando

$$f(n) = g(n) + h(n)$$

Para A^* , além do caminho, mostre o custo total da solução.

import heapq

def ca_estrela (grafo, sh, unico, objetivo):

contador = 0

frontier = [(sh[unico], 0, contador, unico, [unico])] 3

visitados = []

ordem = []

while frontier:

f, ig, -, atual, caminho = heapq.heappop(frontier)

if atual in visitados and visitados[atual] <= ig:

continue

visitados[atual] = ig

ordem.append((atual, ig, sh[atual], f))

if atual == objetivo:

return caminho, ordem, ig

for vizinho, custo in grafo[atual]:

mg = ig + custo

if vizinho not in visitados or visitado

[vizinho] > mg:

contador += 1

heapq.heappush(frontier, (mg + sh[vizinho], mg, contador, vizinho, caminho + [vizinho]))

return None, ordem, None

caminho, ordem, custo-ideal = ca_estrela(grafo, sh, 'S', 'G')

id. execute os três algoritmos para encontrar um caminho entre S e G e complete a tabela:

	Caminho	Custo	Expandido	Último?
BFS	S-A-D-G	9	4	Não
Busca Gulosa	S-B-E-G	8	4	Não
A*	S-C-B-E-G	6	5	Sim

18. Os três algoritmos expandiram os estados na mesma ordem? Justifique?
 Não, os três não expandiram os estados na mesma ordem, cada um usa um critério diferente para escolher o próximo nó, então cada um caminha pelo grafo de um jeito.

19. O algoritmo que expandiu menos estados necessariamente encontrou o caminho de menor custo? Explique.

Não, quem expande menos nós não garante o caminho mais barato. A busca gulosa expandiu os 4, mas seu caminho custou 8. Menos expansões não significa busca mais rápida, não mais barata.

20. Altere o valor de $h(C)$ de 4 para 1. Execute novamente Busca Gulosa e A*. A ordem de expansão mudou? O caminho final mudou? Explique o papel da heurística nos dois algoritmos.

Busca Gulosa

→ Ordem de expansão: antes (S, B, E, G) agora (S, C, B, E, G)

→ Caminho final: não mudou

→ Heurística: decide qual é o caminho

A*

→ Ordem de expansão: não mudou

→ Caminho final: não mudou

→ Heurística: O caminho final continua ótimo